

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 1(b)

Student Name: K. Saranya

Roll No.: A24126552090

1. TITLE

Draw 2D Graphics Using HTML5 Canvas and Style a Webpage Using CSS3

2. AIM

To draw basic 2D shapes and text on an HTML5 canvas using JavaScript, and to style a complete webpage — header, navigation bar, content cards, and footer — using CSS3 properties such as gradients, shadows, and hover effects.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the basic structure and purpose of the `<canvas>` element.
- To obtain a 2D drawing context using `getContext("2d")`.
- To draw a rectangle using `fillRect()`.
- To draw a circle using `arc()` and `fill()`.
- To draw a straight line using `moveTo()`, `lineTo()`, and `stroke()`.
- To render text on a canvas using `fillText()`.
- To style a page header using CSS gradients and text-shadow.
- To create a navigation bar with hover effects using CSS pseudo-classes.
- To lay out content cards using Flexbox.
- To apply box-shadow, border-radius, and transform for visual depth and interactivity.

4. THEORY

The HTML5 `<canvas>` element is simply an empty rectangular region on the page. It has no drawing ability of its own — every shape, line, or piece of text visible inside it is painted using JavaScript, through an object called the drawing context. CSS3 is used, within a `<style>` block in the same file, to style the rest of the page — the header, navigation, cards, and footer — using properties for colour, spacing, shadows, and layout.

The key APIs and properties used in this program are:

- `getContext("2d")` returns the drawing context object that provides all the draw functions used below.

- `fillStyle` sets the colour used by the next filled shape or piece of text.
- `fillRect(x, y, w, h)` draws a filled rectangle at position (x, y) with the given width and height.
- `beginPath()` and `arc()` start a new shape and then draw a circular arc; a full sweep (0 to 2π) makes a complete circle.
- `moveTo()` and `lineTo()` move the pen without drawing, then draw a straight line to the next point; `stroke()` renders it.
- `fillText(text, x, y)` draws text on the canvas, with (x, y) marking the bottom-left of the text baseline.
- `linear-gradient()` blends two colours smoothly across an element's background, used on the header.
- `:hover` applies a style only while the mouse pointer is over an element — used on links, buttons, and cards.
- `box-shadow` and `border-radius` give the cards and header a soft, rounded, elevated appearance.
- `transform: translateY()` shifts a card upward on hover, giving it a gentle lift animation.

The basic flow of the page is:

HTML loads the canvas element → JavaScript gets its 2D context → Draw commands paint shapes onto the canvas → CSS3 rules style the rest of the page

5. ALGORITHM / PROCEDURE

1. Create one HTML file (index.html) that will hold the canvas section, header, navigation bar, cards, and footer, along with an internal `<style>` block for CSS3 rules and a `<script>` block for the canvas drawing.
2. Add a `<canvas>` element with a fixed width and height inside a canvas-section `<div>`.
3. In the `<script>` tag, get a reference to the canvas element using `document.getElementById()`.
4. Call `getContext("2d")` on the canvas to obtain the drawing context.
5. Set `fillStyle` to skyblue and draw a rectangle using `fillRect()`.
6. Begin a new path and draw a full circle using `arc()`, then fill it orange.
7. Begin a new path, use `moveTo()` and `lineTo()` to define a straight horizontal line, and render it using `stroke()`.
8. Set the font and `fillStyle`, then draw the name "Saranya" on the canvas using `fillText()`.
9. Add a semantic header with a gradient background, a navigation bar with three links, three content cards with buttons, and a footer below the canvas section.
10. In the `<style>` block, style the header using `linear-gradient()` and `text-shadow`.
11. Style the navigation links with spacing and a `:hover` colour change.
12. Lay the cards out using Flexbox with `border-radius` and `box-shadow` for depth.
13. Add a `transform: translateY(-5px)` rule on `.card:hover` so each card lifts slightly on hover.
14. Style the buttons with padding, rounded corners, and a hover background change.
15. Save the file and open index.html in a browser to verify the canvas drawing and page styling.

6. PROGRAM

index.html

```

1  <!DOCTYPE html>
2  <html lang="en">
3
4  <head>
5      <meta charset="UTF-8">
6      <title>My Styled Webpage</title>
7      <style>
8          /* ===== Part B: CSS3 Styling ===== */
9
10         .site-header {
11             background: linear-gradient(to right, #4facfe, #00f2fe);
12             /* Gradient background */
13             color: white;
14             font-family: 'Segoe UI', sans-serif;
15             font-size: 30px;
16             text-align: center;
17             padding: 30px;
18             text-shadow: 2px 2px 5px rgba(0, 0, 0, 0.4);
19             /* Text shadow */
20         }
21
22         .site-header p {
23             font-size: 16px;
24             font-weight: normal;
25             margin-top: 8px;
26         }
27
28         .navbar {
29             background-color: #222;
30             text-align: center;
31             padding: 14px;
32         }
33
34         .navbar a {
35             color: #f4f4f4;
36             margin: 0 18px;
37             text-decoration: none;
38             font-family: Arial, sans-serif;
39             font-size: 16px;
40         }
41
42         .navbar a:hover {
43             color: #ff6600;
44             /* Hover effect on nav links */
45         }
46
47         .section-title {
48             text-align: center;
49             font-family: 'Segoe UI', sans-serif;
50             margin-top: 30px;
51             color: #333;
52         }
53
54         .cards {
55             display: flex;
56             justify-content: center;
57             gap: 25px;
58             margin: 25px;
59             flex-wrap: wrap;
60         }
61
62         .card {
63             background-color: #ffffff;
64             border: 2px solid #e0e0e0;
65             border-radius: 14px;
66             /* Rounded corners */
67             padding: 22px;

```

```

68         width: 200px;
69         box-shadow: 4px 4px 12px rgba(0, 0, 0, 0.15);
70         /* Box shadow */
71         text-align: center;
72         font-family: Georgia, serif;
73     }
74
75     .card:hover {
76         transform: translateY(-5px);
77         /* Card lifts on hover */
78     }
79
80     .card h3 {
81         font-size: 19px;
82         color: #4facfe;
83     }
84
85     .card p {
86         font-size: 14px;
87         color: #555;
88     }
89
90     .card button {
91         padding: 8px 18px;
92         border: none;
93         border-radius: 6px;
94         background-color: #4facfe;
95         color: white;
96         cursor: pointer;
97         margin-top: 12px;
98         font-size: 14px;
99     }
100
101     .card button:hover {
102         background-color: #0077cc;
103         /* Hover effect on button */
104     }
105
106     .site-footer {
107         background-color: #222;
108         color: #ccc;
109         text-align: center;
110         padding: 18px;
111         margin-top: 30px;
112         font-family: Arial, sans-serif;
113         font-size: 14px;
114     }
115
116     .canvas-section {
117         text-align: center;
118         margin: 25px 0;
119     }
120
121     canvas {
122         border: 1px solid #000;
123         display: block;
124         margin: 15px auto;
125     }
126 </style>
127 </head>
128
129 <body>
130
131     <!-- ===== Part A: HTML5 Canvas ===== -->
132     <div class="canvas-section">
133         <h2 class="section-title">Canvas Graphics</h2>
134         <p style="font-family: Arial, sans-serif; color:#666;">

```

```

135         A simple rectangle, circle, line, and my name drawn using JavaScript on the HTML5
canvas.
136     </p>
137     <canvas id="myCanvas" width="400" height="250"></canvas>
138 </div>
139
140 <script>
141     const canvas = document.getElementById("myCanvas");
142     const ctx = canvas.getContext("2d"); // 2D drawing context
143
144     ctx.fillStyle = "skyblue";
145     ctx.fillRect(20, 20, 100, 60); // Rectangle
146
147     ctx.beginPath();
148     ctx.arc(220, 60, 40, 0, Math.PI * 2); // Circle
149     ctx.fillStyle = "orange";
150     ctx.fill();
151
152     ctx.beginPath();
153     ctx.moveTo(20, 150);
154     ctx.lineTo(300, 150); // Line
155     ctx.stroke();
156
157     ctx.font = "20px Arial";
158     ctx.fillStyle = "black";
159     ctx.fillText("Saranya", 20, 200); // Name as text
160 </script>
161
162 <!-- ===== Part B: CSS3 Styled Webpage Layout ===== -->
163 <header class="site-header">
164     <h1>My Styled Webpage</h1>
165     <p>Showcasing CSS3 styling with gradients, shadows, and hover effects</p>
166 </header>
167
168 <nav class="navbar">
169     <a href="#">Home</a>
170     <a href="#">About</a>
171     <a href="#">Contact</a>
172 </nav>
173
174 <h2 class="section-title">What I Bring</h2>
175 <div class="cards">
176     <div class="card">
177         <h3>About Me</h3>
178         <p>A student learning web design, currently exploring HTML5, CSS3, and JavaScript
basics.</p>
179         <button>Read More</button>
180     </div>
181     <div class="card">
182         <h3>My Skills</h3>
183         <p>Building structured layouts, styling with CSS3, and adding simple canvas
graphics.</p>
184         <button>View Skills</button>
185     </div>
186     <div class="card">
187         <h3>Get in Touch</h3>
188         <p>Open to feedback and collaboration on small web development projects.</p>
189         <button>Contact Me</button>
190     </div>
191 </div>
192
193 <footer class="site-footer">
194     <p>&copy; 2026 My Styled Webpage | Designed and Coded by Saranya</p>
195 </footer>
196
197 </body>
198
199 </html>

```

7. CODE EXPLANATION

Code	Explanation
<canvas id="myCanvas">	An empty drawing surface; JavaScript is required to paint anything inside it.
getContext("2d")	Returns the 2D drawing context object with all draw functions (fillRect, arc, etc.).
fillStyle	Sets the colour used by the next filled shape or piece of text.
fillRect(x, y, w, h)	Draws a filled rectangle at (x, y) with the given width and height.
beginPath(), arc(), fill()	Start a new shape, define a circular arc, then paint it — used to draw the circle.
moveTo(), lineTo(), stroke()	Move the pen, draw a straight line to a point, then render the line.
fillText(text, x, y)	Draws text on the canvas, with (x, y) as the bottom-left of the text.
linear-gradient(to right, ...)	Blends two colours smoothly across the header's background.
text-shadow	Draws a soft shadow behind text so it stands out against the gradient.
:hover	Applies a style only while the mouse pointer is over a link, button, or card.
display: flex; gap	Arranges the three content cards in a row with even spacing.
box-shadow, border-radius	Give the cards a soft, rounded, elevated ('lifted') appearance.
transform: translateY(-5px)	Lifts a card slightly upward when the mouse hovers over it.

8. TEST CASES AND EXECUTION DATA

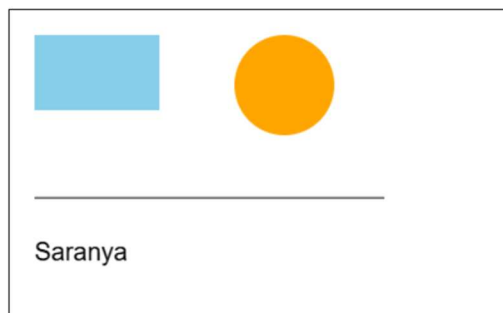
The page was opened in a browser and each drawn shape and styled element was checked individually; the results were recorded below.

Test Case: TC-01
Action: Open index.html in a web browser with default settings
Expected Result: Canvas 2D context paints all required graphics—rectangle, circle, line, and text.
Actual Result: Skyblue box, orange circle, horizontal line, and name text rendered correctly.
Status: Pass

TC-01 Output:

Canvas Graphics

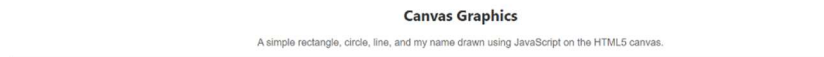
A simple rectangle, circle, line, and my name drawn using JavaScript on the HTML5 canvas.



Test Case: TC-02

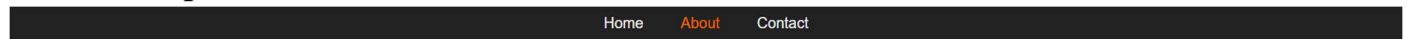
Action: Disable JavaScript in browser settings and reload index.html
Expected Result: Canvas script fails to run; canvas box remains blank.
Actual Result: 2D context failed to initialize; canvas container rendered completely empty.
Status: Fail (Script Execution Disabled)

TC-02 Output:



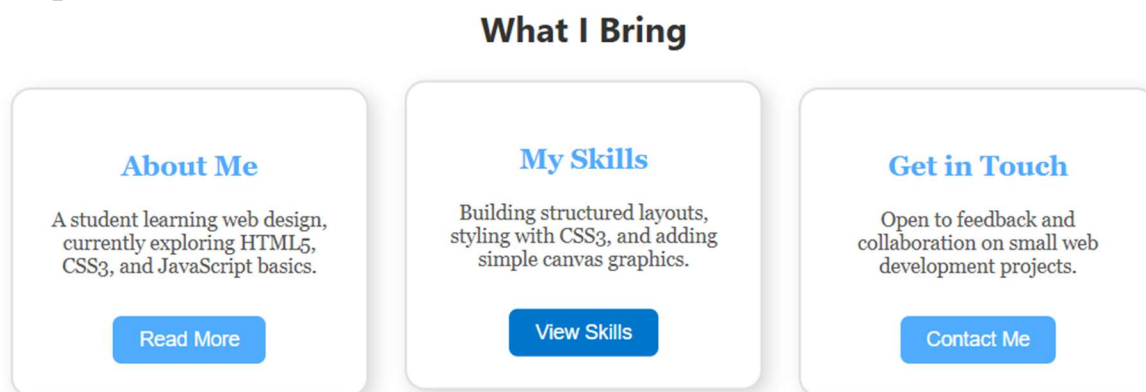
Test Case: TC-03
Action: Hover mouse cursor over navigation bar links (Home, About, Contact)
Expected Result: Navigation text color changes from light grey #f4f4f4 to orange #ff6600.
Actual Result: Link text updated color dynamically on hover.
Status: Pass

TC-03 Output:



Test Case: TC-04
Action: Hover mouse cursor over individual .card containers
Expected Result: CSS rule .card:hover applies transform: translateY(-5px) to lift card vertically.
Actual Result: Card shifted upward smoothly on cursor hover.
Status: Pass

TC-04 Output:

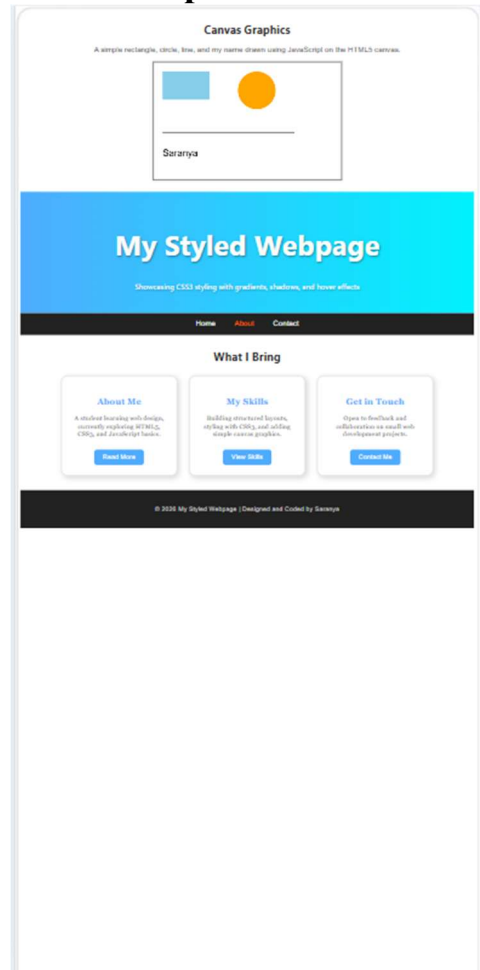


Test Case: TC-05
Action: Tap a content card on a touch-screen device without a physical mouse cursor
Expected Result: Card lift animation (translateY) triggers seamlessly on touch input.

Actual Result: Mobile browser directly registers tap as a click; smooth :hover lift animation does not trigger.(*"Card requires an explicit tap/click to show hover state; continuous touch hover without tapping is not supported on mobile touch screens."*)

Status: Fail (No Mobile Touch Animation Fallback)

TC-05 Output:



Test Case: TC-06

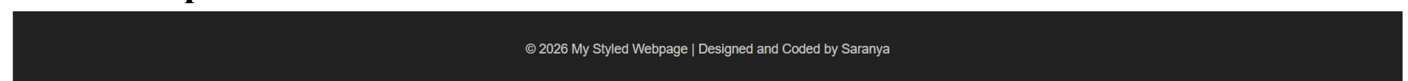
Action: Inspect footer section display at the bottom of the page.

Expected Result: Footer displays dark background #222 with centered copyright text and author credit.

Actual Result: Footer text and background color display as intended.

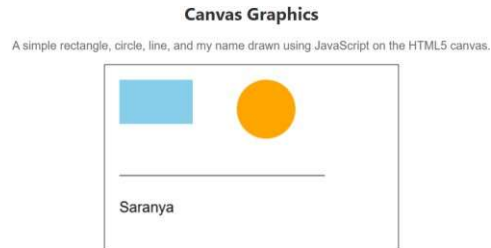
Status: Pass

TC-06 Output:



9. OUTPUT

Output 1: Canvas Graphics — Rectangle, Circle, Line, and Text



Output 2: Styled Header (Gradient) and Navigation Bar



Output 3: Content Cards and Footer



10. RESULT

Thus, canvas graphics — a rectangle, a circle, a line, and text — were successfully drawn using JavaScript, and a complete webpage was styled using CSS3 with a gradient header, hover effects, Flexbox cards, and shadows. All shapes rendered correctly on the canvas, the navigation links and card buttons responded to hover with a colour change, and the cards lifted smoothly on hover, confirming that the page behaves exactly as intended.